Занятие 25.2. using-стейтменты

Если вы часто используете Стандартную библиотеку C++, то постоянное добавление `std::` к используемым объектам может быть несколько утомительным, не правда ли? Язык C++ предоставляет альтернативы в виде using-стейтментов.

Оглавление:

1. Использование «using-объявления»
2. Использование «using-директивы»
3. Пример конфликта с «using-директивой»
4. Область видимости «using-объявления» и «using-директивы»
5. Отмена/замена using-стейтментов

Использование «using-объявления»

Одной из альтернатив является использование **«using-объявления»**. Вот программа «Hello, world!» с «using-объявлением» в строке №5:

```
1 #include <iostream>
2
3 int main()
4 {
5    using std::cout; // "using-объявление" сообщает компилятору, что cout следует обрабатывать, как std::cout
6    cout << "Hello, world!"; // и никакого префикса std:: уже здесь не нужно!
7    return 0;
8 }
```

Строка `using std::cout;` сообщает компилятору, что мы будем использовать объект **cout** из **пространства имен** std. И каждый раз, когда компилятор будет сталкиваться с `cout`, он будет понимать, что это `std::cout`.

Конечно, в этом случае мы не сэкономили много усилий, но в программе, где объекты из пространства имен std используются сотни, если не тысячи раз, «using-объявление» неплохо так экономит время, усилия и улучшает читабельность кода. Также для каждого объекта нужно использовать отдельное «using-объявление» (например, отдельное для `std::cout`, отдельное для `std::cin` и отдельное для `std::endl`).

Хотя этот способ является менее предпочтительным, чем использование префикса `std::`, он все же является абсолютно безопасным и приемлемым.

Использование «using-директивы»

Второй альтернативой является использование **«using-директивы»**. Вот программа «Hello, world!» с «using-директивой» в строке №5:

```
1 #include <iostream>
2
3 int main()
4 {
5    using namespace std; // "using-директива" сообщает компилятору, что мы используем все объекты из пространства имен std!
6    cout << "Hello, world!"; // так что никакого префикса std:: здесь уже не нужно!
7    return 0;
8 }
```

Много разработчиков спорят насчет использования «using-директивы». Так как с её помощью мы подключаем ВСЕ имена из пространства имен std, то вероятность возникновения конфликтов имен значительно возрастает (но все же эта вероятность в глобальном масштабе остается незначительной). `using namespace std;` сообщает компилятору, что мы хотим использовать всё, что находится в пространстве имен std, так что, если компилятор найдет имя, которое не сможет распознать, он будет проверять его наличие в пространстве имен std.

Совет: Старайтесь избегать использования «using-директивы» (насколько это возможно).

Пример конфликта с «using-директивой»

Рассмотрим пример, где использование «using-директивы» создает неопределенность:

```
1  #include <iostream>
2
3  int cout() // объявляем нашу собственную функцию "cout"
4  {
5      return 4;
6  }
7
8  int main()
9  {
10     using namespace std; // делаем std::cout доступным по "cout"
11     cout << "Hello, world!"; // какой cout компилятор здесь должен использовать? Тот, который из пространства имен std или тот
12
13     return 0;
14 }
```

Здесь компилятор не сможет понять, использовать ли ему std::cout или функцию cout(), которую мы определили сами. В результате, получим ошибку неоднозначности. Хоть это и банальный пример, но если бы мы добавили префикс `std::` к cout:

```
1     std::cout << "Hello, world!"; // сообщаем компилятору, что хотим использовать std::cout
```

Или использовали бы «using-объявление» вместо «using-директивы»:

```
1     using std::cout; // сообщаем компилятору, что cout означает std::cout
2     cout << "Hello, world!"; // так что здесь следует использовать std::cout
```

Тогда наша программа была бы без ошибок.

Большинство программистов избегают использования «using-директивы» именно по этой причине. Другие считают это приемлемым до тех пор, пока «using-директива» используется только в пределах отдельных функций (что значительно сокращает масштабы возникновения конфликтов имен).

Области видимости «using-объявления» и «using-директивы»

Если «using-объявление» или «using-директива» используются в блоке, то они применяются только внутри этого блока (по обычным правилам локальной области видимости). Это хорошо, поскольку уменьшает масштабы возникновения конфликтов имен до отдельных блоков. Однако многие начинающие программисты пишут «using-директиву» в глобальной области видимости (вне функции main() или вообще вне любых функций). Этим они *вытаскивают* все имена из пространства имен std напрямую в глобальную область видимости, значительно увеличивая вероятность возникновения конфликтов имен. А это уже не хорошо.

Правило: Никогда не используйте using-стейтменты вне тела функций.

Отмена/замена using-стейтментов

Как только один using-стейтмент был объявлен, его невозможно отменить или заменить другим using-стейтментом в пределах области видимости, в которой он был объявлен. Например:

```
1 int main()
2 {
3     using namespace Boo;
4
5     // Отменить «использование пространства имен Boo» здесь невозможно!
6     // Также нет никакого способа заменить «using namespace Boo» на другой using-стейтмент
7
8     return 0;
9 } // действие using namespace Boo заканчивается здесь
```

Лучшее, что вы можете сделать — это намеренно ограничить область применения using-стейтментов с самого начала, используя правила **локальной области видимости**:

```
1  int main()
2  {
3      {
4          using namespace Boo;
5          // Здесь всё относится к пространству имен Boo::
6      } // действие using namespace Boo заканчивается здесь
7
8      {
9          using namespace Foo;
10         // Здесь всё относится к пространству имен Foo::
11     } // действие using namespace Foo заканчивается здесь
12
13     return 0;
14 }
```

Конечно, всей этой головной боли можно было бы избежать, просто использовав оператор разрешения области видимости (`::`).